

Attention Is All You Need

Gated Recurrent Neural Network (RNN) = Gated Recurrent Unit (GRU) is a specialized deep learning architecture designed to process sequential data (like text, audio, or time-series) while overcoming the memory loss and instability of the conventional RNNs.

Problem: RNNs's problem: used the LSTMs and GRUs (Gated Recurrent Units)
These models compute the hidden states through a recurrence relation.

$$h_t = f(h_{t-1}, x_t)$$

Meaning that each step depend on the previous step = cannot compute h_t without the first computing.

- **Sequential Dependency chain:** each token depend on the previous token, so if a token take 100ms to be generated, then 100 tokens would probably take 10s.
- **Vanishing/Exploding Gradient:** during the [Backpropagation](#), the gradient must travel through every timestep
- **Fixed size bottleneck:** In encoder-decoder RNN, the entire input sequence is compressed into one fixed sized vector (e.g., "The cat sat on the table." --> $[0.3, 0.3423, 0.3254, \dots]$ = Single vector). For the long sequence of text, this create the info loss because all of the information must fit in one single vector.
- **Long-Sequence Dependency Degradation:** to connect token x_1 to the token x_n , information must travel through the $n - step$

Solution that the Attention Mechanism propose

- **No Sequential computation:** the entire QK^T matrix, comparing every token to every other token is computed in one matrix operation.
 - No step depend on previous step --> full GPU parallelized.

$$\begin{bmatrix} \cdot & \cdot & \cdot \\ \cdot & \cdot & \cdot \\ \cdot & \cdot & \cdot \end{bmatrix}$$

Note: The K represent column and the Q represent row

- **Constant path length between any 2 tokens:** Token of position 1 can attend to token at position 1000 with only 1 step.
- **No fixed-sized bottleneck:** the encoder produces a sequence of context-rich vector
 - "The cat sleep" --> $[V - The, V - cat, V - sleep]$
 - Each vector can attend to all other
- **Gradient computation is direct:** because there are no recurrence connection, gradient flow directly from any output position back to any input position without passing through immediate states.

Scaled-Dot Product

$$Attention(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

We use the $\sqrt{d_k}$ to prevent the gradient to vanish in the softmax when the d_k is large cause it will likely push the softmax into region where it has extremely small gradient.

Self-Attention

$$Attention(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

For example:

"I love machine learning"

Each word is converted into an embedding

$[I]$ $[love]$ $[machine]$ $[learning]$

Then each embedding is transformed into the:

- Query (Q) --> $Q = XW_Q$
- Key (T) --> $K = XW_K$
- Value (V) --> $V = XW_V$

Where X is the input embedding matrix

It brighten how words should prioritize/pay attention to other words

For example: "The animal did not cross the street because it was tired."

So when processing "it", self-attention may assign high weight to the "animal", helping the model to understand that "it" refers to the animal rather than the street.

We used it to understand the relationship within a sequence like the grammar, long-range dependencies, conference ("John...he"), content

Cross Attention

It is used in the decoder, and it has 2 sequences, so the first sequence provides the queries, while the other provide the keys and values (e.g., Query -> **decoder state, while the other two are encoder outputs**)

For example:

"The cat sat"

Then the decoder generates:

"Le chat --"

and the Cross-attention scores might be:

- The = 0.05
- cat = 0.15

- $\text{sat} = 0.80$

Then, the decoder focuses mostly on "sat" to generate the next french equivalent of "sat".

Note: the weight of a sequence is cumulative of 1

We use the Cross-Attention to connect two different sequences like Translation, summarization, image captioning (image features $\rightarrow$ text), and speech recognition.

Masked Self-attention

It is the self-attention on steroid designed specifically for the independently generated tasks. So each token of the masked self-attention is prevented from attending to future tokens.

For example: "I love machine learning"

Through the same attention score formula, before applying the softmax, we must modify the attention scores.

Suppose the unmodified attention scores are:

$$[2, 3, 4, 5]$$

So normally

$$\text{softmax}([2, 3, 4, 5])$$

But with the masking:

$$\text{softmax}([2, -\infty, -\infty, -\infty])$$

Then, it becomes

$$[1, 0, 0, 0]$$

So for the "I love machine learning"

When predicting the "machine", the model can attend to "I" and "love" but not "leaning" because this word has not been generated yet.

And we use it to train the model to generate one token at a time without cheating.

Here's the same step-by-step manual computation as the first example, now applied to "i love machine learning".

Mathematical walkthrough

Example: "i love machine learning"

Tokens: i, love, machine, learning $\rightarrow n = 4$, embedding dim $d = 6$, projection dim $d_k = 4$

$$X = \begin{bmatrix} 0.248 & -0.069 & 0.324 & 0.762 & -0.117 & -0.117 \\ 0.790 & 0.384 & -0.235 & 0.271 & -0.232 & -0.233 \\ 0.121 & -0.957 & -0.862 & -0.281 & -0.506 & 0.157 \\ -0.454 & -0.706 & 0.733 & -0.113 & 0.034 & -0.712 \end{bmatrix} \leftarrow i \leftarrow \text{love} \leftarrow \text{machine} \leftarrow \text{learning}$$

Step 1 Project to Q, K, V

$$Q = XW_Q = \begin{bmatrix} 0.074 & -0.861 & -0.461 & -0.184 \\ -0.214 & -0.090 & -0.780 & 0.585 \\ -0.072 & 0.778 & 0.043 & -0.205 \\ 0.858 & 0.060 & 1.012 & -1.579 \end{bmatrix}$$
$$K = XW_K = \begin{bmatrix} -0.321 & -0.402 & -0.380 & -0.526 \\ -0.097 & -0.650 & 0.103 & -0.545 \\ 0.596 & -0.634 & -0.414 & -0.855 \\ -0.235 & 0.334 & -0.384 & -0.347 \end{bmatrix}$$
$$V = XW_V = \begin{bmatrix} -0.279 & 0.061 & 0.277 & 0.241 \\ 0.062 & 0.429 & -1.032 & 0.107 \\ 0.244 & -0.002 & -1.024 & 0.817 \\ 0.184 & 0.035 & 1.193 & 0.844 \end{bmatrix}$$

Note: for the three Weight matrices, they are learned parameter that get started in a random values then get updated via the [Backpropagation](#) and Gradient descent when the model is trained on the data, so that they started to capture the relationship like which words should attend to which.

Step 2: Raw scores QK^T

$$\begin{bmatrix} 0.595 & 0.605 & 0.939 & -0.064 \\ 0.093 & -0.321 & -0.248 & 0.116 \\ -0.198 & -0.382 & -0.379 & 0.332 \\ 0.147 & 0.844 & 1.404 & -0.021 \end{bmatrix}$$

Step 3: Scale by $\sqrt{d_k} = \sqrt{4} = 2$

so that the softmax can produce the result just like the One-hot encoding $[0, 0, 1]$

$$\begin{bmatrix} 0.298 & 0.303 & 0.469 & -0.032 \\ 0.046 & -0.160 & -0.124 & 0.058 \\ -0.099 & -0.191 & -0.189 & 0.166 \\ 0.073 & 0.422 & 0.702 & -0.011 \end{bmatrix}$$

Step 4: Perform Softmax on each row

Take row 1 (query = "i") as a worked example:

$$e^{0.298} = 1.347, e^{0.303} = 1.354, e^{0.469} = 1.599, e^{-0.032} = 0.969$$

$$\text{sum} = 1.347 + 1.354 + 1.599 + 0.969 = 5.269$$

$$A_{i,:} = \left[\frac{1.347}{5.269}, \frac{1.354}{5.269}, \frac{1.599}{5.269}, \frac{0.969}{5.269} \right] = [0.256, 0.257, 0.304, 0.184]$$

Doing this for all rows:

$$A = \begin{bmatrix} 0.256 & 0.257 & 0.304 & 0.184 \\ 0.273 & 0.222 & 0.230 & 0.276 \\ 0.242 & 0.221 & 0.221 & 0.316 \\ 0.192 & 0.272 & 0.360 & 0.176 \end{bmatrix} \leftarrow i \leftarrow \text{love} \leftarrow \text{machine} \leftarrow \text{learning}$$

Note: (each row sums to 1)

Step 5: Output = AV

Worked example for row 1 ("i"):

$$0.256 \begin{bmatrix} -0.279 \\ 0.061 \\ 0.277 \\ 0.241 \end{bmatrix} + 0.257 \begin{bmatrix} 0.062 \\ 0.429 \\ -1.032 \\ 0.107 \end{bmatrix} + 0.304 \begin{bmatrix} 0.244 \\ -0.002 \\ -1.024 \\ 0.817 \end{bmatrix} + 0.184 \begin{bmatrix} 0.184 \\ 0.035 \\ 1.193 \\ 0.844 \end{bmatrix} = \begin{bmatrix} 0.052 \\ 0.132 \\ -0.286 \\ 0.492 \end{bmatrix}$$

Full output matrix (final self-attention representations):

$$\text{Attention}(Q, K, V) = \begin{bmatrix} 0.052 & 0.132 & -0.286 & 0.492 \\ 0.044 & 0.121 & -0.060 & 0.510 \\ 0.058 & 0.120 & -0.011 & 0.529 \\ 0.083 & 0.134 & -0.385 & 0.518 \end{bmatrix} \leftarrow i \leftarrow \text{love} \leftarrow \text{machine} \leftarrow \text{learning}$$

Each row is the new contextualized vector for that word — a weighted blend of every token's value vector, weighted by that row of A .

Add & Norm

- **Residual connection (Add):** instead of passing the sublayer directly forward, the input is added back to the output

$$\text{Output} = \text{Sub-layer}(x) + x$$

Note: same idea as the ResNets that let the gradient flow unchanged during the backpropagation

- **Layer normalization (Norm)**

$$\text{Output} = \text{LayerNorm}(\text{Sub-layer}(x) + x)$$

- **LayerNorm** = rescales the values across the feature dimension to have mean of 0 and variance of 1 to keep the activation in a stable range as they passed through multiple layers, preventing the activation exploding or shrinkage.
 - **Mean of 0** = Center the data around 0, meaning that activations are not all positive or negative, they are all balanced. Weight in the next layer can then push the value either directly freely, without fighting the built-in bias
 - **Variance of 1** = ensuring that all features have the same scale. Without this, one dimension with value like [500, 440, 324] would dominate over another with [0.1, 0.3, 0.4], even if both carry the same equally useful information

$$\hat{x} = \frac{x - \mu}{\sigma}$$

--> Training is faster, and gradient don't explode or vanish and then the optimizer use the same learning rate effectively

The Encoder and Decoder

- **Encoder:** it reads the input sequence and converts it into a set of rich vector representation (embeddings) that capture the meaning and context of each input token.
 - **For example:** "The cat sleep"
--> $[V - The, V - cat, V - sleep]$
 - The encoder process all the words simultaneously (using the self-attention)
 - Each vector contains information not only about that word but also about the surrounding words.
 - The encoder mainly uses the **self-attention**, where every input token attends to every other input token.
For example:
 - **Input:** The cat sat
"sat" attends to: "the", "cat", "sat"
--> To help understand the context.
- **Decoder:** it generates the output sequence one token at a time. And at every step, it uses:
 - **Masked Self-attention** to look only at previously generated output tokens
 - **For example:** It has generated "Le chat" then it wants to predict the next word and can only attend to the "Le", "chat", but not the future words.
 - **Cross-attention** to look at the encoder's output and decide which input words are most relevant for generating the next output token.

Note: not all Transformer-based models have both an encoder and decoder.

- **Encoder-only models (e.g., BERT):** Used for understanding tasks like classification, sentiment analysis, and question answering
- **Decoder-only models (e.g., GPT):** Used for the text generation. These models do not have a separate encoder; they rely only on the masked self-attention over the prompt and previous generated text
- **Encoder-Decoder models (e.g., T5):** Used for tasks that transform one sequence into another, such as translation, summarization, and paraphrasing.

And FYI, these encoders and decoders have nothing to do with the Autoencoder. Both pose different purposes.

Multi-Head Attention

We use the Multi-Head attention so that each head can attend to different representation subspaces simultaneously to capture local, global, syntactic, and semantic pattern in parallel.

Position-Wise FFN

In a transformer block, each layer has two main sub-components:

1. **Multi-head self-attention** = let the token exchange information with each other
2. **Position-wise Feed Forward Network (FFN)** = processes each token independently

Note: The FFN is not part of the attention mechanism itself, but it is a separate sublayer that comes right after attention in each transformer block.

Position-wise = Attention mixes information across position (tokens attend to other tokens).

So the FFN does the opposite as it applies the same transformation independently to each position/token, with no interaction between tokens, So if you have a sequence of length n , the exact same 2-layer neural network is applied to each of the n token vectors separately in parallel.

$$FFN(x) = \max(0, xW_1 + b_1)W_2 + b_2$$

- W_1 = expands from d_{model} to a larger inner dimension d_{ff} (often 4x larger, e.g., 512 $\rightarrow$ 2048)
- ReLU (or GeLU in modern models) applies a non-linearity
- W_2 = Projects back down from d_{ff} to d_{model}

Note: Expanding the dimension gives the network a more bird-view to work with the documents, picking up more pattern than before.

$$Linear \rightarrow Activation(ReLU) \rightarrow Linear$$

Positional Encoding

So as the self-attention has no sense of order because there is no recurrence and convolution.

$$PE(pos, 2i) = \sin\left(\frac{pos \times d_{model}}{10000^{2i}}\right)$$

$$PE(pos, 2i + 1) = \cos\left(\frac{pos \times d_{model}}{10000^{2i}}\right)$$

Where i is the dimension index.

Then the positional encoding vector (same size as the token embedding) is simply added to each token embedding before feeding into the transformer.

Why Sinusoids?

- Different mechanism oscillate at different frequency
 - Low i - slow wave (capture coarse position)
 - High i - fast wave (capture fine-grained position)
- The model can represent **relative positions** through linear combinations of these signals
- It generates to sequence lengths not seen during training

- Simply put "the nearby position produce similar encoding"

Architecture

```

Token embedding
  --> Positional Encoding
    --> Multi-head attention
      --> Add & Norm
        --> FFN (Position wise)
          --> Add & Norm
            --> Output

```

More on the Pre-norm and Post-norm

It describe where the Layer Norm is placed in the transformer block relative to the sub-layer (attention and FFN) and skip/residual connection (think of it as a single continuous wire running end-to-end through the whole network)

(a) **Post-norm:** the original transformer.

$$\text{Output} = \text{LayerNorm}(\text{Sub-layer}(x) + x)$$

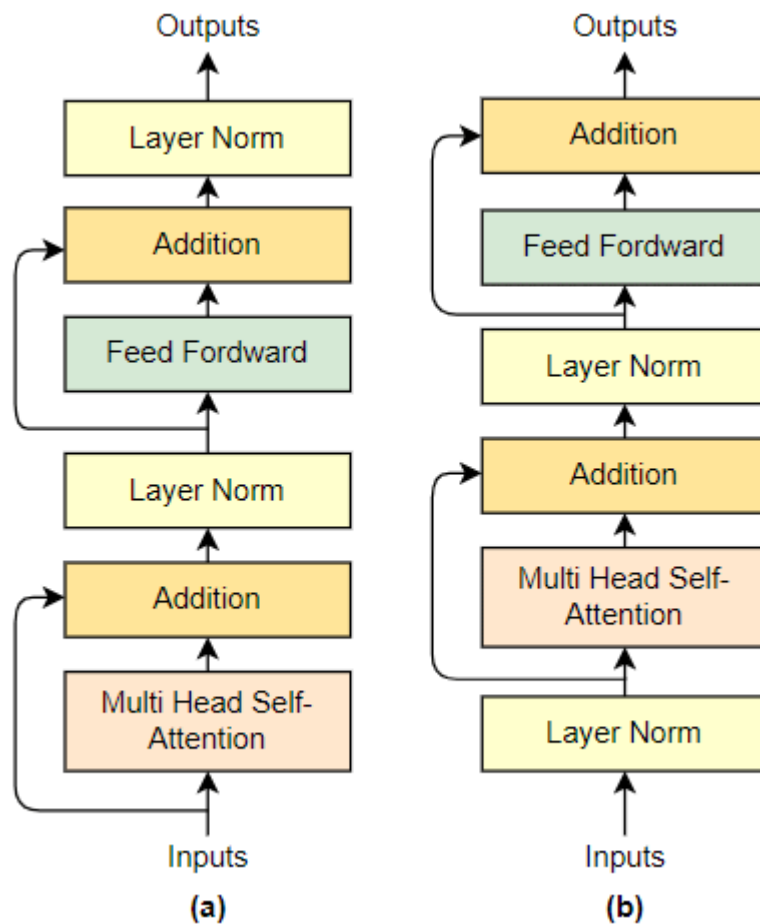
In post-norm, gradients have to pass *through* the LayerNorm on every residual path, which can cause gradients to shrink or explode in deep networks, especially early in training. Post-norm transformers typically need careful learning-rate warmup to avoid divergence.

- It need the learning rate warmth up, harder for the deep net
- (b) **Pre-norm:** the modified/modernize transformer.

$$\text{Output} = x + \text{Sub-layer}(\text{LayerNorm}(x))$$

In pre-norm, the residual stream is a clean, unimpeded highway so the x flows straight through addition without being renormalized. This means gradients can flow directly back through the residual connections almost unchanged, making very deep networks much easier to train and less sensitive to warmup schedules.

- It is very stable, scale



Limitation

LIMITATION	WHY IT IS A PROBLEM	LATER IMPROVEMENTS
Quadratic attention	Memory and computation grow as $O(n^2)$ with sequence length.	Sparse attention, linear attention, FlashAttention, long-context architectures
Fixed positional encoding	Sinusoidal encoding is not always optimal.	Learned embeddings, relative positions, RoPE, ALiBi
No recurrence	Processes all tokens together but does not inherently model streaming or persistent state.	Transformer-XL, recurrent memory mechanisms, state-space models
Large data requirement	High-capacity models need massive datasets to generalize well.	Better pretraining objectives, data curation, scaling laws
Computational cost	Training large Transformers requires substantial compute and memory.	Mixed precision, model parallelism, efficient kernels, Mixture-of-Experts
Limited context window	Original models were trained with relatively short sequence lengths.	Extended-context training and efficient attention variants